

Tutorial Sheet 3

Section - A

1. Define stack. What are the applications of stack?

→ A stack is a widely used linear data structure in modern computers in which insertions and deletions of an element can occur only at one end i.e., top of the stack. It follows the Last-In, First-out (LIFO) principle, meaning that the last element added to the stack is the first one to be removed.

The applications of stack are -

- (i) Function call management
- (ii) Backtracking
- (iii) Memory Management
- (iv) Expression Evaluation
- (v) Browser History

2. List out the basic operations that can be performed on a stack.

→ The basic operations that can be performed on a stack:

- (i) Push: Adding an element to the top of the stack.
- (ii) Pop: Removing the top element from the stack.
- (iii) Peek or Top: Viewing the top element without removing it.
- (iv) IsEmpty: Checking if the stack is empty.
- (v) Size: Returns the size of stack.

3. What is overflow and underflow in stack?

→ (i) Overflow: Stack overflow occurs when you try to push an element onto a stack that is already full.

and has reached its maximum capacity. It is a common issue in stack-based data structures and it can lead to errors in program.

(ii) Underflow: Stack underflow happens when you try to pop (remove) an element from an empty stack i.e. there are no elements left to remove. Like overflow, underflow can also lead to error in program.

4. What is the postfix expression for given postfix expression the corresponding infix expression?

$$a + b * c + (d * e)$$

$$\rightarrow abc * + de * +$$

5. The equivalent infix expression for given postfix expression $abc + de / * -$ is ?

$$\rightarrow \text{Infix} \Rightarrow a(b+c)(d/e) * -$$

$$\Rightarrow a(b+c) * (d/e) -$$

$$\Rightarrow a - (b+c) * (d/e)$$

Ans.

Section - B

1. Differentiate iteration and recursion with example.

Property	Recursion	Iteration
1. Definition	Function call itself	A set of instructions repeatedly executed
2. Application	For functions	For loops
3. Termination	Through base case, where there will be no function call	When the termination condition for the iterator ceases to be satisfied
4. Usage	Used when code size needs to be small, & time	used when time complexity needs to be balanced against

Teacher's Signature.....

	complexity is not an issue.	an expanded code size.
5. Code Size	Smaller code size.	Larger code size.
6. Time Complexity	Very high (generally exponential) time complexity.	Relatively lower time complexity (generally polynomial - logarithmic).
7. Space Complexity	It is higher than iterations.	It is lower.
8. Stack	Here the stack is used to store local variables when the function is called.	Stack is not used.
9. Speed	Execution is slow since it has the overhead of maintaining and updating the stack.	Normally, it is faster than recursion as it doesn't utilize the stack.
10. Memory	Uses more memory.	Uses less memory.
11. Overhead	Possesses overhead of repeated function calls.	No overhead as there are no function calls.
12. Infinite repetition	If the recursive function does not meet the termination condition or the base case is not defined or is never reached then it leads to a stack overflow error and there is a chance that the system may crash in infinite recursion.	If the control condition of the iteration statement never becomes false or the control variable does not reach the termination value, then it will cause infinite loop. In the infinite loop, it uses the CPU cycles again and again.

Example:

```
#include <iostream>

int sum(int n){
    int result = 0;
    for (int i = 1; i <= n; i++){
        result += i;
    }
}
```


Example of Recursion:

```
#include <iostream>

int sum_recursive(int n) {
    if (n == 1) {
        return 1;
    }
    else {
        return n + sum_recursive(n-1);
    }
}

int main() {
    std::cout << sum_recursive(5) << std::endl;
    return 0;
}
```

Example of iteration:

```
#include <iostream>

int sum_iterative(int n) {
    int result = 0;
    for (int i = 1; i <= n; ++i) {
        result += i;
    }
    return result;
}

int main() {
    std::cout << sum_iterative(5) <<
    std::endl;
    return 0;
}
```

2. Write an algorithm Convert the following infix expressions into prefix expressions:

(a) $(A * B) + (C / D) - (D + E)$

(b) $((A - B) + D / ((E + F) * G))$

→ (a) $(A * B) + (C / D) - (D + E)$

prefix: Reverse the expression i.e. $(E + D) - (D / C) + (B * A)$

Expression	Stack	Prefix
(	(	
E	(	E
+	(+	E
D	(+	ED
)		ED +
-	-	ED +
(	-((	ED +
D	-(	ED + D
/	-(/	ED + D

C	- C /	ED + DC
)	-	ED + DC /
+	- +	ED + DC /
(	- + (	ED + DC /
B	- + C	ED + DC / B
*	- + (*	ED + DC / B
A	- + (* A	ED + DC / B A
)	- + (	ED + DC / B A *
		ED + DC / B A * + -

Reverse the expression $\rightarrow$ $- + * AB / CD + DE$ Any

b) $((A-B) + D / ((E+F) * G))$

Reverse in expression: $((G * (F+E)) / D + (B-A))$

Expression

Stack

Prefix

(	(	
(	((	
G	((G	G
*	((* G	G
(	((* (G	G
F	((* (GF	GF
+	((* (+ GF	GF
E	((* (+ GFE	GFE
)	((* GFE +	GFE +
)	(GFE + *	GFE + *
/	(/ GFE + *	GFE + *
D	(/ GFE + * D	GFE + * D
+	(+ GFE + * D /	GFE + * D /
(	(+ (GFE + * D /	GFE + * D /
B	(+ (GFE + * D / B	GFE + * D / B
-	(+ (- GFE + * D / B	GFE + * D / B

A

(+ (-

G F E + * D / B A

)

(+

G F E + * D / B A -

)

G F E + * D / B A - +

Rewrite the expression: $+ - A B / D * + E F G$ Ans.

3. Differentiate b/w Infix, Prefix and Postfix Notations.

→ 1. Infix Notation: It's the most common notation used in mathematics and is familiar to most people. It is of the form $\langle \text{operand} \rangle \langle \text{operator} \rangle \langle \text{operand} \rangle$. As the name suggests, the operator is fixed inside between the operands. Such expressions are easy to understand and evaluate for human beings. However, computer finds it difficult to parse - information is needed ~~to~~ about operator precedence and associativity rules and brackets which override these rules.

Eg. $(A * B) / C$.

2. Prefix Notation: The prefix notation as the name suggests has the operator placed before the operand is specified. It is of the form $\langle \text{operator} \rangle \langle \text{operand} \rangle \langle \text{operand} \rangle$. It works entirely in same manner as postfix expression. For evaluation, we evaluate it from left to right.

Prefix expressions are also called as polish notation.

Eg. $* + A B - C D$

3. Postfix Notation: In this notation, the operator placed right after the two operands. It is of the form $\langle \text{operand} \rangle \langle \text{operand} \rangle \langle \text{operator} \rangle$. It itself determines the precedence of operators i.e. the operator which occurs first operates on the operands. Eg. $A B + C D - *$

Section C

1. Explain tower of Hanoi with stack ADT.

→ The Tower of Hanoi is a mathematical puzzle. It consists of three towers (pegs) and a set of disks of different sizes. The disks can be stacked onto the pegs, with the largest disk at the bottom and smallest disk at the top. The objective is to move the entire stack from one tower known as source tower, to another tower known as target tower, using the third tower called the auxiliary tower as temporary storage. The puzzle follows three simple rules:

- (i) Only one disk can be moved among the towers at any given time.
- (ii) Only the "top" disk can be removed.
- (iii) No larger disk can be placed over a smaller disk.

Implementation of tower of Hanoi problem in C++ using stacks:

```
#include <iostream>
```

```
#include <stack>
```

```
#include <bits/stdc++.h>
```

```
void TowerOfHanoi (int numDisks, std::stack<int> source,  
std::stack<int> target, std::stack<int> auxiliary)
```

```
{ int totalMoves = pow(2, numDisks) - 1;
```

```
for (int move = 1; move <= totalMoves; move++) {
```

```
    if (move % 3 == 1) {
```

```
        if (!source.empty() && (auxiliary.empty() || source.top() <  
auxiliary.top())) {
```

```
            auxiliary.push (source.top());
```



```
source.pop();
```

```
std::cout << "Move disk from auxiliary source to auxiliary" <<
std::endl;
```

```
}
```

```
else {
```

```
source.push(auxiliary.top());
```

```
auxiliary.pop();
```

```
std::cout << "Move disk from auxiliary to source" << std::endl;
```

```
}
```

```
}
```

```
else if (move < 3 == 2) {
```

```
if (!source.empty() && (target.empty() || source.top() <
target.top())) {
```

```
target.push(source.top());
```

```
source.pop();
```

```
std::cout << "Move disk from source to target" << std::endl;
```

```
} else { source.push(target.top());
```

```
target.pop();
```

```
std::cout << "Move disk from target to source" << std::endl;
```

```
}
```

```
} else { if (!auxiliary.empty() && (target.empty() || auxiliary.top() <
target.top())) {
```

```
target.push(auxiliary.top());
```

```
auxiliary.pop();
```

```
std::cout << "Move disk from auxiliary to target" << std::endl;
```

```
} else { auxiliary.push(target.top());
```

```
target.pop();
```

```
std::cout << "Move disk from target to auxiliary" << std::endl;
```

```
}
```

```
} } } }
```



```
int main () { int numDisks ;  
    std::cout << "Enter the number of disks : " ;  
    std::cin >> numDisks ;  
    std::stack <int> source, target, auxiliary ;  
    for (int i = numDisks ; i >= 1 ; i--) {  
        source.push (i) ;  
    }  
    Tower of Hanoi (numDisks , source, target , auxiliary) ;  
    return 0 ;  
}
```

The time complexity of Tower of Hanoi solution using stack is $O(2^n)$, where n is the number of disks and the space complexity of Tower of Hanoi using stacks is $O(n)$.

2. Write an algorithm to convert an infix expression to its corresponding postfix Expression, using a stack.

→ Algorithm :

- (i) First start scanning the operator from left to right.
- (ii) If the scanned character is an operand, output it i.e., print it.

(iii) Else

- If the precedence and associativity of the scanned operator are greater than the precedence and associativity of the operator in the stack [or the stack is empty or the stack contains a '('], Then push it in the stack. ['^' operator is right associative and other operators like '+', '-', '*', and '/' are left-associative].

- Check especially for a condition when the operator at the top of the stack and the scanned operator both are '^'. In this

condition, the precedence of the scanned operator is higher due to its right associativity. So it will be pushed into the operator stack.

- In all the other cases when the top of the operator stack is the same as the scanned operator, then pop the operator from the stack because of left associativity due to which the scanned operator has less precedence.
- Else, Pop all the operators from the stack which are greater than or equal to in precedence than that of the scanned operator.
- After doing that push the scanned operator to the stack.
(If you encounter parenthesis while popping then stop there and push the scanned operator in the stack.)
- (iv) If the scanned character is a '(' push it to the stack.
- (v) If the scanned character is a ')' pop the stack and output it until a '(' is encountered, and discard both the parentheses.
- (vi) Repeat steps until the infix expression is scanned.
- (vii) Once the scanning is over, Pop the stack and add the operations in the postfix expression until it is not empty.
- (viii) Finally, print the postfix expression.

3. Convert following infix expression into postfix expression using a stack:

$$((a+b)/d) - ((e-f)+g)$$

→ Expression	Stack	Postfix
(	(	
(	((	
a	((	a
+	((+	a

b	((+	ab
)	(	ab+
/	(/	ab+
d	(/	ab+d
)		ab+d/
-	-	ab+d/
(	-(	ab+d/
(	-(	ab+d/
e	-(	ab+d/e
-	-(	ab+d/e
f	-(	ab+d/ef
)	-(	ab+d/ef-
+	-(	ab+d/ef-
g	-(	ab+d/ef-g
)	-	ab+d/ef-g+
		ab+d/ef-g+-

Postfix expression: $ab+d/ef-g+-$